

DSA Revision Notes

Topic 10 — Dynamic Programming (Complete)

1D · 2D · Interval · Knapsack · Bitmask · Tree DP · Digit DP
LCS · LIS · Edit Distance · Matrix Chain · Partition · 30+ C++ examples

14

Sections

35+

C++ Examples

12

DP Tables

35+

LeetCode Problems

TOC Table of Contents**1. What is DP? — Identifying DP Problems**

- Overlapping subproblems · Optimal substructure
- Top-down vs bottom-up · Memoisation template
- State identification: the 5-step framework

2. 1D Dynamic Programming

- Fibonacci / Climbing Stairs · House Robber
- Jump Game · Coin Change · Decode Ways · Word Break

3. 2D Dynamic Programming

- Unique Paths · Minimum Path Sum
- Maximal Square · Dungeon Game

4. Classic Sequence DP

- Longest Common Subsequence (LCS) — $O(mn)$ table
- Longest Increasing Subsequence (LIS) — $O(n \log n)$
- Edit Distance (Levenshtein)

5. Knapsack Patterns

- 0/1 Knapsack · Unbounded Knapsack
- Partition Equal Subset Sum · Target Sum
- Coin Change II · Last Stone Weight II

6. Interval DP

- Matrix Chain Multiplication · Burst Balloons
- Palindrome Partitioning · Strange Printer

7. Bitmask DP

- Travelling Salesman (Held-Karp) · Shortest Superstring
- Smallest Sufficient Team

8. Tree DP

- House Robber III · Binary Tree Cameras
- Diameter · Distribute Coins

9. Digit DP

- Framework · Count numbers with digit property
- Non-decreasing digits · At most K zeros

10. String DP

- Longest Palindromic Subsequence · Distinct Subsequences
- Interleaving String · Regular Expression Matching

11. Stock Trading Problems

- Best Time I–V (all variants) · State machine DP

12. DP Optimisations

- Space optimisation (rolling array) · Convex Hull Trick
- Divide & Conquer DP · Knuth's Optimisation

13. Greedy vs DP — When to Choose

14. Complexity Cheat Sheet & LeetCode Map

1 What is DP? — Identifying & Solving DP Problems

1.1 Two Necessary Conditions for DP

A problem can be solved with DP if and only if it has both: (1) **Optimal Substructure** — the optimal solution to the problem contains optimal solutions to subproblems. (2) **Overlapping Subproblems** — the same subproblems are solved repeatedly in a naive recursive solution. If only the first holds, use greedy. If only the second, use memoisation without DP semantics.

■ DP Signal Recognition

SIGNAL 1: 'Find the maximum/minimum...' → likely DP (optimise over choices)

SIGNAL 2: 'Count the number of ways...' → likely DP (add subproblem counts)

SIGNAL 3: 'Is it possible to...' → likely DP (boolean subproblems)

SIGNAL 4: Problem involves sequences, subsets, partitions, or grids

SIGNAL 5: Exponential brute-force has a polynomial DP solution (polynomial state space)

NOT DP: greedy works when a locally optimal choice is always globally optimal (exchange arg).

1.2 Top-Down vs Bottom-Up

Top-Down (Memoisation): start from the original problem, recurse toward base cases, cache results. Natural to write, easy to reason about. Uses $O(\text{depth})$ stack space. **Bottom-Up (Tabulation):** start from base cases, fill the table iteratively. No recursion overhead, easier to optimise space. Preferred in production.

2 1D Dynamic Programming

2.1 Climbing Stairs (LC 70) — Easy

You can climb 1 or 2 steps. How many ways to reach step n ? Each step = sum of ways from 1 or 2 steps below.

Climbing Stairs

$dp[i] = dp[i-1] + dp[i-2]$

Base: $dp[1] = 1, dp[2] = 2$

Answer: $dp[n]$

C++

```
1  int climbStairs(int n) {
2      if (n <= 2) return n;
3      int a=1, b=2;
4      for (int i=3; i<=n; i++) { int c=a+b; a=b; b=c; }
5      return b;
6  }
7  // Time: O(n)   Space: O(1)
```

2.2 House Robber (LC 198) — Medium

Cannot rob two adjacent houses. At each house, choose: rob it (skip prev) or skip it (keep prev max). Classic 'include or exclude' 1D DP.

House Robber

$dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$

$= \max(\text{skip house } i, \text{rob house } i)$

Base: $dp[0] = \text{nums}[0], dp[1] = \max(\text{nums}[0], \text{nums}[1])$

C++

```

1  int rob(vector<int>& nums) {
2      int n = nums.size();
3      if (n == 1) return nums[0];
4      int prev2 = nums[0], prev1 = max(nums[0], nums[1]);
5      for (int i=2; i<n; i++) {
6          int cur = max(prev1, prev2 + nums[i]);
7          prev2 = prev1; prev1 = cur;
8      }
9      return prev1;
10 }
11 // Time: O(n)   Space: O(1)
12
13 // LC 213 House Robber II (circular): run rob() twice
14 // Case 1: rob nums[0..n-2]   Case 2: rob nums[1..n-1]
15 // Answer: max(case1, case2)

```

2.3 Coin Change (LC 322) — Medium

Minimum number of coins to make amount. Unbounded use of each coin. For each amount a from 1 to target, try all coins: $dp[a] = \min(dp[a], dp[a - \text{coin}] + 1)$.

Coin Change

$dp[a] = \min \text{ over all coins } c \text{ where } c \leq a: dp[a - c] + 1$

Base: $dp[0] = 0$ (0 coins to make amount 0)

Init: $dp[1..amount] = \text{INF}$

C++

```

1  int coinChange(vector<int>& coins, int amount) {
2      vector<int> dp(amount+1, INT_MAX);
3      dp[0] = 0;
4      for (int a=1; a<=amount; a++) {
5          for (int c : coins) {
6              if (c <= a && dp[a-c] != INT_MAX)
7                  dp[a] = min(dp[a], dp[a-c]+1);
8          }
9      }
10     return dp[amount] == INT_MAX ? -1 : dp[amount];
11 }
12 // Time: O(amount * coins)   Space: O(amount)

```

2.4 Decode Ways (LC 91) — Medium

Count ways to decode a digit string. Each character or pair of characters maps to a letter (1-26). $dp[i] = \text{ways to decode } s[0..i-1]$. Extend by 1 digit (valid: $s[i-1] \neq '0'$) or 2 digits (valid: 10-26).

C++

```

1  int numDecodings(string s) {
2      int n = s.size();
3      vector<int> dp(n+1, 0);
4      dp[0] = 1; // empty prefix = 1 way
5      dp[1] = s[0] != '0' ? 1 : 0; // single digit
6      for (int i=2; i<=n; i++) {
7          int one = stoi(s.substr(i-1,1)); // last 1 digit
8          int two = stoi(s.substr(i-2,2)); // last 2 digits
9          if (one >= 1) dp[i] += dp[i-1];
10         if (two >= 10 && two <= 26) dp[i] += dp[i-2];
11     }
12     return dp[n];
13 }
14 // Time: O(n)   Space: O(n) → O(1) with two variables

```

2.5 Word Break (LC 139) — Medium

Can string *s* be segmented using words from *wordDict*? *dp[i]* = true if *s*[0..*i*-1] can be segmented. Try all words as the last segment.

C++

```

1  bool wordBreak(string s, vector<string>& wordDict) {
2      unordered_set<string> dict(wordDict.begin(), wordDict.end());
3      int n = s.size();
4      vector<bool> dp(n+1, false);
5      dp[0] = true; // empty prefix is valid
6      for (int i=1; i<=n; i++) {
7          for (int j=0; j<i; j++) {
8              if (dp[j] && dict.count(s.substr(j, i-j))) {
9                  dp[i] = true; break;
10             }
11         }
12     }
13     return dp[n];
14 }
15 // Time: O(n^2 * L)   L=avg word length   Space: O(n)

```


3 2D Dynamic Programming

3.1 Unique Paths (LC 62) — Medium

Count paths from top-left to bottom-right in an $m \times n$ grid (only right or down moves). $dp[i][j] = dp[i-1][j] + dp[i][j-1]$. All cells in first row and first column = 1.

Unique Paths (2D)

$dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Base: $dp[0][j] = 1$ for all j (top row)

$dp[i][0] = 1$ for all i (left col)

Answer: $dp[m-1][n-1]$

C++

```
1  int uniquePaths(int m, int n) {
2      vector<int> dp(n, 1); // space-optimised: single row
3      for (int i=1; i<m; i++)
4          for (int j=1; j<n; j++)
5              dp[j] += dp[j-1]; // dp[j] = from above, dp[j-1] = from left
6      return dp[n-1];
7  }
8  // Time: O(m*n)   Space: O(n) – rolling array
9  // LC 63 Unique Paths II: skip cells where grid[i][j] == 1 (obstacle)
```

3.2 Minimum Path Sum (LC 64) — Medium

Minimum sum path from top-left to bottom-right. $dp[i][j] = grid[i][j] + \min(dp[i-1][j], dp[i][j-1])$.

C++

```
1  int minPathSum(vector<vector<int>>& grid) {
2      int m=grid.size(), n=grid[0].size();
3      for (int i=0; i<m; i++) {
4          for (int j=0; j<n; j++) {
5              if (i==0 && j==0) continue;
6              else if (i==0) grid[i][j] += grid[i][j-1];
7              else if (j==0) grid[i][j] += grid[i-1][j];
8              else grid[i][j] += min(grid[i-1][j], grid[i][j-1]);
9          }
10     }
11     return grid[m-1][n-1];
12 }
13 // Time: O(m*n)   Space: O(1) – modify in-place
```

3.3 Maximal Square (LC 221) — Medium

Find largest square of 1s in a binary matrix. $dp[i][j]$ = side length of largest square with bottom-right corner at (i,j) . Key: $dp[i][j] = \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1$ when $matrix[i][j] == '1'$.

C++

```
1  int maximalSquare(vector<vector<char>>& matrix) {
2      int m=matrix.size(), n=matrix[0].size(), maxlen=0;
3      vector<vector<int>> dp(m, vector<int>(n,0));
4      for (int i=0; i<m; i++) {
5          for (int j=0; j<n; j++) {
6              if (matrix[i][j]=='1') {
7                  if (i==0 || j==0) dp[i][j]=1;
8                  else dp[i][j] = min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]}) + 1;
9                  maxlen = max(maxlen, dp[i][j]);
10             }
11         }
12     }
13     return maxlen * maxlen;
14 }
15 // Time: O(m*n)   Space: O(m*n) → O(n) with rolling array
```

4 Classic Sequence DP

4.1 Longest Common Subsequence (LC 1143) — Medium

LCS of two strings is the longest subsequence present in both. $dp[i][j]$ = LCS of $s1[0..i-1]$ and $s2[0..j-1]$. If characters match: extend; else take max of skipping one character from either string.

Longest Common Subsequence

```
if s1[i-1] == s2[j-1]: dp[i][j] = dp[i-1][j-1] + 1
else: dp[i][j] = max(dp[i-1][j], dp[i][j-1])
Base: dp[0][j] = dp[i][0] = 0 (empty string)
```

		B	D	C	A	B
A	0	0	0	0	0	0
B	0	1	1	1	1	1
C	1	1	1	1	2	2
B	1	1	2	2	3	3
D	1	2	2	2	3	3
A	1	2	2	3	3	4
B	1	2	2	3	4	4

LCS table for 'ABCBDB' and 'BDCAB' — answer = $dp[7][5] = 4$

C++

```
1 int longestCommonSubsequence(string s1, string s2) {
2     int m=s1.size(), n=s2.size();
3     vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
4     for (int i=1; i<=m; i++)
5         for (int j=1; j<=n; j++)
6             if (s1[i-1]==s2[j-1]) dp[i][j]=dp[i-1][j-1]+1;
7             else dp[i][j]=max(dp[i-1][j], dp[i][j-1]);
8     return dp[m][n];
9 }
10 // Time: O(m*n) Space: O(m*n) → O(n) with rolling row
```

4.2 Longest Increasing Subsequence (LC 300) — Medium

$O(n^2)$ DP: $dp[i]$ = length of LIS ending at index i . For each $j < i$ where $nums[j] < nums[i]$: $dp[i] = \max(dp[i], dp[j]+1)$. **$O(n \log n)$ Patience Sorting:** maintain 'tails' array where $tails[k]$ = smallest tail of all IS of length $k+1$. Binary search for insertion point.

C++

```

1  //  $O(n^2)$  DP
2  int lisDP(vector<int>& nums) {
3      int n=nums.size(), ans=1;
4      vector<int> dp(n,1);
5      for (int i=1; i<n; i++)
6          for (int j=0; j<i; j++)
7              if (nums[j]<nums[i]) { dp[i]=max(dp[i],dp[j]+1); ans=max(ans,dp[i]); }
8      return ans;
9  }
10 // Time:  $O(n^2)$    Space:  $O(n)$ 
11
12 //  $O(n \log n)$  – Patience Sorting
13 int lisNLogN(vector<int>& nums) {
14     vector<int> tails;
15     for (int x : nums) {
16         auto it = lower_bound(tails.begin(), tails.end(), x);
17         if (it == tails.end()) tails.push_back(x);
18         else *it = x; // replace with smaller tail
19     }
20     return tails.size();
21 }
22 // Time:  $O(n \log n)$    Space:  $O(n)$ 
23 // tails is NOT the actual LIS – it's the structure for length calculation only

```

4.3 Edit Distance (LC 72) — Hard

Minimum insert/delete/replace operations to convert word1 to word2. $dp[i][j]$ = edit distance between word1[0..i-1] and word2[0..j-1]. If chars match: no cost; else: 1 + min(replace, delete, insert).

Edit Distance

```

if w1[i-1] == w2[j-1]: dp[i][j] = dp[i-1][j-1]
else: dp[i][j] = 1 + min(dp[i-1][j-1], // replace
                        dp[i-1][j],      // delete from w1
                        dp[i][j-1])      // insert into w1

```

C++

```
1  int minDistance(string w1, string w2) {
2      int m=w1.size(), n=w2.size();
3      vector<vector<int>> dp(m+1, vector<int>(n+1,0));
4      for (int i=0; i<=m; i++) dp[i][0]=i;    // delete all of w1
5      for (int j=0; j<=n; j++) dp[0][j]=j;    // insert all of w2
6      for (int i=1; i<=m; i++)
7          for (int j=1; j<=n; j++)
8              if (w1[i-1]==w2[j-1]) dp[i][j]=dp[i-1][j-1];
9              else dp[i][j]=1+min({dp[i-1][j-1], dp[i-1][j], dp[i][j-1]});
10     return dp[m][n];
11 }
12 // Time: O(m*n)    Space: O(m*n) → O(n) rolling
```

5 Knapsack Patterns

5.1 0/1 Knapsack — Each Item Used At Most Once

$dp[j]$ = maximum value achievable with capacity exactly j (or at most j). Process items one by one. For each item, iterate capacity **right to left** (prevents using the same item twice — since we read from the unupdated left).

0/1 Knapsack

$dp[j] = \max(dp[j], dp[j - \text{weight}[i]] + \text{value}[i])$ if $j \geq \text{weight}[i]$

Iterate j from W down to $\text{weight}[i]$ (RIGHT to LEFT!)

Base: $dp[0..W] = 0$

C++

```
1  int knapsack01(vector<int>& wt, vector<int>& val, int W) {
2      int n = wt.size();
3      vector<int> dp(W+1, 0);
4      for (int i=0; i<n; i++)
5          for (int j=W; j>=wt[i]; j--) // RIGHT to LEFT
6              dp[j] = max(dp[j], dp[j-wt[i]] + val[i]);
7      return dp[W];
8  }
9  // Time: O(n*W)   Space: O(W)   — pseudo-polynomial
```

5.2 Unbounded Knapsack — Each Item Used Unlimited Times

Same as 0/1 but iterate capacity **left to right** — this allows the same item to be used multiple times in the same pass.

C++

```
1  int knapsackUnbounded(vector<int>& wt, vector<int>& val, int W) {
2      vector<int> dp(W+1, 0);
3      for (int i=0; i<(int)wt.size(); i++)
4          for (int j=wt[i]; j<=W; j++) // LEFT to RIGHT
5              dp[j] = max(dp[j], dp[j-wt[i]] + val[i]);
6      return dp[W];
7  }
8  // Time: O(n*W)   Space: O(W)
```

5.3 Partition Equal Subset Sum (LC 416) — Medium

Split array into two subsets with equal sum. Equivalent to: can we pick a subset summing to $\text{total}/2$? 0/1 Knapsack with $\text{target} = \text{sum}/2$. $dp[j] = \text{true}$ if subset summing to j exists.

C++

```
1 bool canPartition(vector<int>& nums) {
2     int total = 0; for (int x:nums) total+=x;
3     if (total%2) return false;
4     int target = total/2;
5     vector<bool> dp(target+1, false);
6     dp[0] = true;
7     for (int x : nums) // 0/1 knapsack
8         for (int j=target; j>=x; j--) // right to left
9             dp[j] = dp[j] || dp[j-x];
10    return dp[target];
11 }
12 // Time: O(n*sum) Space: O(sum)
```

5.4 Target Sum (LC 494) — Medium

Assign + or - to each number. Count assignments that sum to target. Transform: let P = sum of + numbers, N = sum of - numbers. $P - N = \text{target}$, $P + N = \text{sum} \rightarrow P = (\text{sum} + \text{target})/2$. Count subsets summing to P.

C++

```
1 int findTargetSumWays(vector<int>& nums, int target) {
2     int total=0; for (int x:nums) total+=x;
3     if ((total+target)%2 || abs(target)>total) return 0;
4     int goal = (total+target)/2;
5     vector<int> dp(goal+1, 0);
6     dp[0]=1;
7     for (int x : nums) // 0/1 count knapsack
8         for (int j=goal; j>=x; j--) // right to left
9             dp[j] += dp[j-x];
10    return dp[goal];
11 }
12 // Time: O(n*sum) Space: O(sum)
```

5.5 Coin Change II (LC 518) — Medium

Count number of ways to make amount using unlimited coins. Unbounded knapsack — iterate coins outer, amount inner left-to-right.

C++

```
1 int change(int amount, vector<int>& coins) {
2     vector<int> dp(amount+1, 0);
3     dp[0]=1;
4     for (int c : coins) // unbounded: coin outer
5         for (int j=c; j<=amount; j++) // left to right
6             dp[j] += dp[j-c];
7     return dp[amount];
8 }
9 // Time: O(amount * coins) Space: O(amount)
10 // KEY: if you swap loops (amount outer, coins inner), you COUNT PERMUTATIONS
11 // e.g. [1,2] and [2,1] are counted separately → Combination Sum IV problem
```


C++

```

1  int maxCoins(vector<int>& nums) {
2      nums.insert(nums.begin(), 1); nums.push_back(1); // sentinels
3      int n = nums.size();
4      vector<vector<int>> dp(n, vector<int>(n, 0));
5      for (int len=2; len<n; len++) {
6          for (int i=0; i+len<n; i++) {
7              int j=i+len;
8              for (int k=i+1; k<j; k++) {
9                  dp[i][j] = max(dp[i][j],
10                     dp[i][k] + nums[i]*nums[k]*nums[j] + dp[k][j]);
11              }
12          }
13      }
14      return dp[0][n-1];
15  }
16  // Time: O(n^3)   Space: O(n^2)

```

6.3 Palindrome Partitioning II (LC 132) — Hard

Minimum cuts to partition string into palindromes. Precompute isPalin[i][j] using expand-around-centre or DP. dp[i] = min cuts for s[0..i].

C++

```

1  int minCut(string s) {
2      int n=s.size();
3      // Precompute palindrome table
4      vector<vector<bool>> pal(n, vector<bool>(n, false));
5      for (int i=n-1; i>=0; i--)
6          for (int j=i; j<n; j++)
7              pal[i][j] = (s[i]==s[j]) && (j-i<=2 || pal[i+1][j-1]);
8      // dp[i] = min cuts for s[0..i]
9      vector<int> dp(n, 0);
10     for (int i=1; i<n; i++) {
11         if (pal[0][i]) { dp[i]=0; continue; } // whole prefix is palindrome
12         dp[i] = i; // worst case: cut before every character
13         for (int j=1; j<=i; j++)
14             if (pal[j][i]) dp[i]=min(dp[i], dp[j-1]+1);
15     }
16     return dp[n-1];
17 }
18 // Time: O(n^2)   Space: O(n^2)

```

7 Bitmask DP

7.1 Bitmask DP — When & How

Use bitmask DP when: $n \leq 20$ (typically), and the state requires tracking a **subset** of elements. State: $dp[mask]$ where bit i of $mask = \text{'element } i \text{ has been used/visited'}$. 2^n states $\times$ n transitions = $O(n \cdot 2^n)$ time, $O(2^n)$ space. Iteration order: enumerate all 2^n masks in increasing order (smaller subsets filled first).

■ Bitmask Operations Cheatsheet

Check if bit i is set: $mask \& (1 \ll i)$

Set bit i : $mask \mid (1 \ll i)$

Clear bit i : $mask \& \sim(1 \ll i)$

Toggle bit i : $mask \wedge (1 \ll i)$

Number of set bits: `__builtin_popcount(mask)`

Iterate over all subsets of $mask$: `for(int sub=mask; sub; sub=(sub-1)&mask)`

All n bits set: $(1 \ll n) - 1$

7.2 Travelling Salesman Problem — Held-Karp $O(n^2 \cdot 2^n)$

$dp[mask][i]$ = minimum cost to visit all cities in $mask$, ending at city i . Start from city 0. At each step, try visiting any unvisited city next.

TSP Held-Karp

$dp[mask][i] = \min_{j \in mask, j \neq i}:$

$dp[mask \wedge (1 \ll i)][j] + dist[j][i]$

Base: $dp[1 \ll 0][0] = 0$ (start at city 0)

Answer: $\min_{i \neq 0} dp[(1 \ll n) - 1][i] + dist[i][0]$

C++

```

1  int tsp(vector<vector<int>>& dist) {
2      int n = dist.size();
3      int FULL = (1<<n)-1;
4      // dp[mask][i] = min cost to visit all cities in mask, ending at i
5      vector<vector<int>> dp(1<<n, vector<int>(n, INT_MAX/2));
6      dp[1][0] = 0; // start at city 0, mask = {0} = 0b0001
7      for (int mask=1; mask<=FULL; mask++) {
8          for (int u=0; u<n; u++) {
9              if (!(mask & (1<<u)) || dp[mask][u] == INT_MAX/2) continue;
10             for (int v=0; v<n; v++) {
11                 if (mask & (1<<v)) continue; // already visited
12                 int newMask = mask | (1<<v);
13                 dp[newMask][v] = min(dp[newMask][v], dp[mask][u]+dist[u][v]);
14             }
15         }
16     }
17     int ans = INT_MAX;
18     for (int u=1; u<n; u++)
19         ans = min(ans, dp[FULL][u] + dist[u][0]);
20     return ans;
21 }
22 // Time: O(n^2 * 2^n)   Space: O(n * 2^n)

```

7.3 Smallest Sufficient Team (LC 1125) — Hard

Find the smallest team such that collectively they cover all required skills. Bitmask DP: $dp[mask]$ = min team to achieve skill set 'mask'. For each person, update all masks that include their skills.

C++

```
1  vector<int> smallestSufficientTeam(  
2      vector<string>& req_skills, vector<vector<string>>& people) {  
3      int m = req_skills.size();  
4      unordered_map<string, int> skillIdx;  
5      for (int i=0; i<m; i++) skillIdx[req_skills[i]]=i;  
6      // Encode each person's skills as a bitmask  
7      vector<int> pSkill(people.size(), 0);  
8      for (int i=0; i<(int)people.size(); i++)  
9          for (auto& s:people[i]) pSkill[i]|=(1<<skillIdx[s]);  
10     int FULL=(1<<m)-1;  
11     vector<long long> dp(FULL+1, LLONG_MAX);  
12     vector<long long> parent(FULL+1, -1);  
13     dp[0]=0;  
14     for (int mask=0; mask<FULL; mask++) {  
15         if (dp[mask]==LLONG_MAX) continue;  
16         for (int i=0; i<(int)people.size(); i++) {  
17             int newMask = mask|pSkill[i];  
18             if (dp[mask]+1<dp[newMask]) {  
19                 dp[newMask]=dp[mask]+1;  
20                 parent[newMask]=((long long)mask<<20)|i;  
21             }  
22         }  
23     }  
24     // Reconstruct team  
25     vector<int> res;  
26     int mask=FULL;  
27     while (mask) { long long p=parent[mask];  
28         res.push_back(p&((1<<20)-1)); mask=p>>20; }  
29     return res;  
30 }  
31 // Time: O(n * 2^m)    Space: O(2^m)
```


C++

```
1 // State: 0=not covered, 1=covered no camera, 2=has camera
2 int cameras = 0;
3 int dfs(TreeNode* node) {
4     if (!node) return 1; // null nodes considered 'covered'
5     int L = dfs(node->left), R = dfs(node->right);
6     if (L==0 || R==0) { cameras++; return 2; } // child uncovered → place camera
7     if (L==2 || R==2) return 1; // child has camera → this node covered
8     return 0; // both children covered but no camera → report uncovered
9 }
10 int minCameraCover(TreeNode* root) {
11     if (dfs(root)==0) cameras++; // root itself not covered
12     return cameras;
13 }
14 // Time: O(n) Space: O(h)
```

9 Digit DP

9.1 Digit DP Framework

Count integers in $[0, N]$ satisfying some digit property. State: (position, tight, leading_zero, ...other_constraints). **tight**: whether the number is still bounded by N's digits. **leading_zero**: whether we've placed a non-zero digit yet.

■ Digit DP State Variables

pos: current digit position being filled (0 = most significant)
 tight: true if all digits so far match N's prefix (limits next digit to $\leq N[pos]$)
 leading_zero: true if no non-zero digit placed yet (handles leading zeros)
 extra state: any problem-specific constraint (digit sum, count of specific digit, etc.)

Transition: for each digit d from 0 to (tight ? $N[pos]$: 9):
 call $dfs(pos+1, tight \&\& (d == N[pos]), leading_zero \&\& (d == 0), \dots update\ state \dots)$

C++

```
1 // Count numbers in [1, n] with digit sum divisible by k
2 int countDivisibleDigitSum(int n, int k) {
3     string s = to_string(n);
4     int len = s.size();
5     // dp[pos][sum mod k][tight]
6     vector<vector<array<int,2>>> memo(
7         len, vector<array<int,2>>(k, {-1,-1}));
8     function<int(int,int,bool)> dp = [&](int pos, int rem, bool tight) -> int {
9         if (pos == len) return rem == 0 ? 1 : 0;
10        int& ref = memo[pos][rem][tight];
11        if (ref != -1) return ref;
12        int limit = tight ? (s[pos]-'0') : 9;
13        int res = 0;
14        for (int d=0; d<=limit; d++)
15            res += dp(pos+1, (rem+d)%k, tight && (d==limit));
16        return ref = res;
17    };
18    return dp(0, 0, true);
19 }
20 // Time: O(len * k * 10)   Space: O(len * k)
```

9.2 LC 1012 — Numbers With Repeated Digits (Hard)

Count integers in $[1, N]$ with at least one repeated digit. Use complement: $N - \text{count}(\text{no repeated digits})$.

C++

```
1  int numDupDigitsAtMostN(int n) {
2      string s = to_string(n+1); // count [1, n] → [0, n+1)
3      int len = s.size(), total = 0;
4      // Count numbers with no repeated digits using digit DP + permutations
5      // memo[pos][mask of used digits][tight]
6      vector<vector<array<int,2>>> memo(len, vector<array<int,2>>(1<<10,{-1,-1}));
7      function<int(int,int,bool,bool)> dp=
8          [&](int pos,int mask,bool tight,bool started)->int{
9              if(pos==len) return started?1:0;
10             auto& ref=memo[pos][mask][tight];
11             if(ref!=-1) return ref;
12             int lim=tight?(s[pos]-'0'):9,res=0;
13             for(int d=0;d<=lim;d++){
14                 if(!started&&d==0){ res+=dp(pos+1,mask,tight&&d==lim,false); continue;}
15                 if(mask&(1<<d)) continue; // digit already used
16                 res+=dp(pos+1,mask|(1<<d),tight&&d==lim,true);
17             }
18             return ref=res;
19         };
20     return n - dp(0,0,true,false) + 1; // complement
21 }
22 // Time: O(len * 2^10 * 10)   Space: O(len * 2^10)
```


10 String DP

10.1 Longest Palindromic Subsequence (LC 516) — Medium

$dp[i][j]$ = LPS of $s[i..j]$. If $s[i]==s[j]$: $dp[i][j] = dp[i+1][j-1] + 2$. Else: $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$. Fill by increasing length.

```
C++
1  int longestPalindromeSubseq(string s) {
2      int n=s.size();
3      vector<vector<int>> dp(n, vector<int>(n,0));
4      for (int i=0; i<n; i++) dp[i][i]=1; // single char is palindrome
5      for (int len=2; len<=n; len++) {
6          for (int i=0; i+len-1<n; i++) {
7              int j=i+len-1;
8              if (s[i]==s[j]) dp[i][j]=dp[i+1][j-1]+2;
9              else dp[i][j]=max(dp[i+1][j], dp[i][j-1]);
10         }
11     }
12     return dp[0][n-1];
13 }
14 // Time: O(n^2)   Space: O(n^2)
```

10.2 Distinct Subsequences (LC 115) — Hard

Count distinct subsequences of s that equal t . $dp[i][j]$ = #ways $s[0..i-1]$ contains $t[0..j-1]$ as subsequence.

```
C++
1  int numDistinct(string s, string t) {
2      int m=s.size(), n=t.size();
3      vector<vector<long long>> dp(m+1, vector<long long>(n+1,0));
4      for (int i=0; i<=m; i++) dp[i][0]=1; // empty t matched 1 way
5      for (int i=1; i<=m; i++)
6          for (int j=1; j<=n; j++) {
7              dp[i][j]=dp[i-1][j]; // skip s[i-1]
8              if (s[i-1]==t[j-1]) dp[i][j]+=dp[i-1][j-1]; // use s[i-1]
9          }
10     return (int)dp[m][n];
11 }
12 // Time: O(m*n)   Space: O(m*n) → O(n) rolling
```

10.3 Regular Expression Matching (LC 10) — Hard

Match string s against pattern p with $.$ (any char) and $*$ (0 or more of preceding). $dp[i][j]$ = true if $s[0..i-1]$ matches $p[0..j-1]$.

C++

```
1  bool isMatch(string s, string p) {
2      int m=s.size(), n=p.size();
3      vector<vector<bool>> dp(m+1, vector<bool>(n+1,false));
4      dp[0][0]=true;
5      // Handle patterns like a*, a*b*, a*b*c* matching empty string
6      for (int j=2;j<=n;j++) if(p[j-1]=='*') dp[0][j]=dp[0][j-2];
7      for (int i=1;i<=m;i++) {
8          for (int j=1;j<=n;j++) {
9              if (p[j-1]=='*') {
10                 dp[i][j]=dp[i][j-2]; // use 0 occurrences of p[j-2]
11                 if (p[j-2]=='.' || p[j-2]==s[i-1])
12                     dp[i][j]=dp[i][j] || dp[i-1][j]; // use 1+ occurrences
13             } else if (p[j-1]=='.' || p[j-1]==s[i-1]) {
14                 dp[i][j]=dp[i-1][j-1];
15             }
16         }
17     }
18     return dp[m][n];
19 }
20 // Time: O(m*n)   Space: O(m*n)
```

11 Stock Trading Problems — State Machine DP

11.1 State Machine for All Stock Problems

All stock problems can be modelled as a state machine with states: **held** (currently holding stock) and **notHeld** (not holding stock). Transition: buy (notHeld $\rightarrow$ held), sell (held $\rightarrow$ notHeld), rest (stay). Each variant adds constraints (cooldown, fee, k transactions).

C++

```
1  int maxProfitK(int k, vector<int>& prices) {
2      int n=prices.size();
3      if (k >= n/2) { // effectively unlimited
4          int res=0;
5          for(int i=1;i<n;i++) res+=max(0,prices[i]-prices[i-1]);
6          return res;
7      }
8      // dp[t][0] = max profit with at most t txns, not holding
9      // dp[t][1] = max profit with at most t txns, holding
10     vector<vector<int>> dp(k+1,vector<int>(2,0));
11     for (int t=1;t<=k;t++) dp[t][1]=INT_MIN/2;
12     for (int price:prices)
13         for (int t=k;t>=1;t--) {
14             dp[t][0]=max(dp[t][0], dp[t][1]+price); // sell
15             dp[t][1]=max(dp[t][1], dp[t-1][0]-price); // buy
16         }
17     return dp[k][0];
18 }
19 // Time: O(n*k)   Space: O(k)
```

12 DP Optimisations

12.1 Space Optimisation — Rolling Array

Many 2D DP problems only depend on the previous row or last two rows. Replace the 2D table with 1 or 2 1D arrays. Key: fill order must ensure we don't overwrite values we still need.

C++

```
1 // LCS: from  $O(m \cdot n)$  to  $O(n)$  space
2 int lcsSPACE(string s1, string s2) {
3     int m=s1.size(), n=s2.size();
4     vector<int> dp(n+1,0), prev(n+1,0);
5     for (int i=1; i<=m; i++) {
6         fill(dp.begin(), dp.end(), 0);
7         for (int j=1; j<=n; j++)
8             if (s1[i-1]==s2[j-1]) dp[j]=prev[j-1]+1;
9             else dp[j]=max(prev[j], dp[j-1]);
10        swap(dp, prev);
11    }
12    return prev[n];
13 }
14 // Space:  $O(n)$  instead of  $O(m \cdot n)$ 
```

12.2 Convex Hull Trick (CHT)

Optimises DP recurrences of the form: $dp[i] = \min \text{ over } j: f(j) + g(i) \cdot h(j)$. Maintain a lower convex hull of lines $y = h(j) \cdot x + f(j)$. Query at $x = g(i)$ gives the minimum. $O(n \log n)$ or $O(n)$ with monotone queries.

C++

```
1 // CHT with monotone queries (Li Chao Tree alternative)
2 // dp[i] = min over j<i: dp[j] + cost(j,i)
3 // If cost = (j+1) * a[i], this is a linear function of j
4 struct Line { long long m, b; long long eval(long long x){ return m*x+b; } };
5
6 struct CHT {
7     deque<Line> hull;
8     bool bad(Line l1, Line l2, Line l3) {
9         return (__int128)(l3.b-l1.b)*(l1.m-l2.m)
10             <= (__int128)(l2.b-l1.b)*(l1.m-l3.m);
11     }
12     void add(Line l) {
13         while (hull.size()>=2 && bad(hull[hull.size()-2],hull.back(),l))
14             hull.pop_back();
15         hull.push_back(l);
16     }
17     long long query(long long x) { // min query, monotone increasing x
18         while (hull.size()>=2 && hull[0].eval(x)>=hull[1].eval(x))
19             hull.pop_front();
20         return hull[0].eval(x);
21     }
22 };
23 // Time: O(n) amortised with monotone queries
24 // Use Li Chao Tree for non-monotone queries: O(n log n)
```

13 Greedy vs DP — When to Choose

13.1 Decision Guide

Both greedy and DP make decisions based on subproblems, but they differ fundamentally: **Greedy** makes a locally optimal choice and never reconsiders. **DP** considers all options and takes the globally optimal one. Greedy is faster ($O(n \log n)$ vs $O(n^2)$) but only correct when the greedy choice property holds.

Problem	Greedy?	Why / Why Not
Activity Selection	Yes	Earliest finish time choice is provably optimal
Fractional Knapsack	Yes	Take highest value-per-weight fraction first
0/1 Knapsack	No	Local best item may block better combination
Coin Change (standard coins)	No	Only works for canonical systems (US coins); need DP generally
Shortest Path (Dijkstra)	Yes	Greedy min-distance extraction is provably optimal
Huffman Coding	Yes	Always merge two least frequent — provably optimal
LCS	No	No greedy choice exists; must try all alignments
Interval Scheduling Maximisation	Yes	Sort by end time, always pick earliest finish
Interval Partitioning	Yes	Sort by start time, assign to any compatible room
Jump Game I (reachable?)	Yes	Track maximum reachable index greedily
Jump Game II (min jumps)	Yes	BFS-like greedy works; also solvable with DP
Coin Change (min coins)	No	Greedy fails for arbitrary denominations

Table 2: Greedy vs DP decision guide

14 Complexity Cheat Sheet & LeetCode Map

14.1 DP Pattern Complexity Summary

DP Pattern	Time	Space	Key Recurrence
1D Fibonacci-style	$O(n)$	$O(1)$	$dp[i] = f(dp[i-1], dp[i-2])$
1D Knapsack 0/1	$O(n \cdot W)$	$O(W)$	$dp[j] = \max(dp[j], dp[j-w]+v)$ — R to L
1D Unbounded Knapsack	$O(n \cdot W)$	$O(W)$	$dp[j] = \max(dp[j], dp[j-w]+v)$ — L to R
2D Grid DP	$O(m \cdot n)$	$O(n)$	$dp[i][j] = f(dp[i-1][j], dp[i][j-1])$
LCS / Edit Distance	$O(m \cdot n)$	$O(n)$	$dp[i][j] = \text{match or max(skip one)}$
LIS ($O(n \log n)$)	$O(n \log n)$	$O(n)$	Binary search on tails array
Interval DP	$O(n^3)$	$O(n^2)$	$dp[i][j] = \text{min/max over split } k$
Tree DP	$O(n)$	$O(h)$	dfs returns subtree state tuple
Bitmask DP	$O(n \cdot 2^{\text{len}})$	$O(n \cdot 2^{\text{len}})$	$dp[\text{mask}][i] = \text{over subsets}$
Digit DP	$O(\text{len} \cdot 10 \cdot \text{state})$	$O(\text{len} \cdot \text{state})$	dfs(pos, tight, state)
String DP	$O(m \cdot n)$	$O(m \cdot n)$	Varies; usually 2D table
Stock Machine DP	$O(n \cdot k)$	$O(k)$	State: (held/notHeld, k txns)
Matrix Chain	$O(n^3)$	$O(n^2)$	$dp[i][j] = \text{min cost to multiply chain}$

Table 3: DP pattern complexity overview

14.2 LeetCode Problem Map

#	Problem	Pattern	Diff
10	Regular Expression Matching	String DP 2D	Hard
32	Longest Valid Parentheses	1D DP / Stack	Hard
44	Wildcard Matching	String DP 2D	Hard
62	Unique Paths	2D Grid DP	Medium
64	Minimum Path Sum	2D Grid DP	Medium
70	Climbing Stairs	1D Fibonacci	Easy
72	Edit Distance	String DP 2D	Hard
91	Decode Ways	1D DP	Medium

115	Distinct Subsequences	String DP 2D	Hard
123	Best Time to Buy/Sell Stock III	State Machine DP	Hard
132	Palindrome Partitioning II	Interval DP	Hard
139	Word Break	1D DP + HashSet	Medium
152	Maximum Product Subarray	1D DP min/max	Medium
188	Best Time to Buy/Sell Stock IV	State Machine K	Hard
198	House Robber	1D DP include/skip	Medium
213	House Robber II	1D DP circular	Medium
221	Maximal Square	2D DP	Medium
300	Longest Increasing Subsequence	LIS $O(n \log n)$	Medium
309	Best Time with Cooldown	State Machine DP	Medium
312	Burst Balloons	Interval DP	Hard
322	Coin Change	Unbounded Knapsack	Medium
337	House Robber III	Tree DP	Medium
416	Partition Equal Subset Sum	0/1 Knapsack Bool	Medium
494	Target Sum	0/1 Count Knapsack	Medium
516	Longest Palindromic Subsequence	Interval DP	Medium
518	Coin Change II	Unbounded Count KS	Medium
714	Best Time with Fee	State Machine DP	Medium
740	Delete and Earn	1D House Robber style	Medium
847	Shortest Path Visiting All Nodes	Bitmask BFS DP	Hard
968	Binary Tree Cameras	Tree DP 3-state	Hard
1143	Longest Common Subsequence	LCS 2D DP	Medium
1125	Smallest Sufficient Team	Bitmask DP	Hard
1312	Minimum Insertion Steps for Palindrome	Interval DP	Hard
1510	Stone Game IV	1D Game Theory DP	Hard

Table 4: 33 LeetCode problems — Dynamic Programming

■ Golden Rules — Dynamic Programming

1. Always define $dp[i]$ precisely BEFORE writing the recurrence. Bad state = wrong solution.
2. 0/1 Knapsack: iterate capacity RIGHT to LEFT. Unbounded: LEFT to RIGHT. This is the ONLY difference.
3. LIS in $O(n \log n)$: binary search on 'tails' array. tails is NOT the actual LIS — only its length is correct.
4. Interval DP: outer loop = LENGTH (2 to n), inner loops = start i , end $j = i + \text{len} - 1$. Never swap.
5. Bitmask DP: iterate masks in increasing order (smaller subsets guaranteed processed first).
6. Digit DP: 'tight' flag limits the current digit to $N[\text{pos}]$. Once tight=false, all digits 0-9 allowed.
7. Stock problems: model as state machine (held/notHeld). Add constraints per variant.
8. Space optimisation: 2D DP that uses only prev row $\rightarrow O(n)$. Use rolling array.
9. Top-down vs bottom-up: same complexity. Use top-down for natural recursion, bottom-up for space opt.
10. If greedy fails on any example, switch to DP. If DP state space is exponential (2^n), try Bitmask DP.